

Lab1-Report

St ID:25303050027

Name:高宁

Github ID:Jacky-Ning-yyqz

个人仓库链接: <https://github.com/Jacky-Ning-yyqz/DataLab>

终端截图: `./check_ops.py bits.c & ./btest`

```
jackyning@jackyning:~/DataLab$ ./btest
Score  Rating  Errors  Function
1      1        0      signMask
2      2        0      bitXor
3      3        0      negativePart
4      4        0      copyByteWithin
4      4        0      logicalShift
4      4        0      swapNibblePairs
4      4        0      secondLowestZeroBit
5      5        0      oddParity
5      5        0      rotateRightBits
5      5        0      roundEvenPow2
5      5        0      midpointTowardFirst
7      7        0      isBetweenEitherOrder
7      7        0      mul5Sat
7      7        0      classifyAdd3
7      7        0      floatScaleThreeHalves
10     10        0      floatRoundEven
10     10        0      float_i2f
10     10        0      bitCount
10     10        0      bitReverse
Total points: 110/110
```

```
jackyning@jackyning:~/DataLab$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 11/24 operators
midpointTowardFirst: 20/32 operators
isBetweenEitherOrder: 28/48 operators
mul5Sat: 19/30 operators
classifyAdd3: 30/52 operators
floatScaleThreeHalves: 57/60 operators
floatRoundEven: 37/65 operators
float_i2f: 30/40 operators
bitCount: 28/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

P1~P19 函数实现思路:

P1 signMask — 构造高位掩码

要求返回 0x80000000, 但常量只能是 0-255, 无法直接书写。注意到该值即"第 31 位为 1", 故用左移构造 $1 \ll 31$ 。常量 1、31 不计入操作数, 实际只用了 1 个操作符。

P2 bitXor — 德摩根定律

要求仅用 $\sim$ 和 $\&$ 实现异或, $|$ 与 $\wedge$ 均被禁用。利用异或的语义「至少一个为 1, 且不同时为 1」, 有 $x \wedge y = (x | y) \& \sim(x \& y)$; 再用德摩根定律把 $|$ 转为 $\&$ 与 $\sim$ ($x | y = \sim(\sim x \& \sim y)$), 最终得 $\sim(\sim x \& \sim y) \& \sim(x \& y)$, 共 7 个操作数。

P3 negativePart — 符号掩码与掩码选择

需实现 $x < 0 ? -x : 0$, 但不能用比较、if 和负号。分三步: ① 用算术右移 $x \gg 31$ 得到符号掩码 (负数为全 1, 非负为全 0); ② 用补码性质 $-x = \sim x + 1$ 求负; ③ 用 $\text{mask} \& \text{value}$ 完成二选一——掩码全 1 时得 $\sim x + 1$, 全 0 时得 0, 无需额外分支。边界上 INT_MIN 的 $\sim x + 1$ 回绕为自身, 恰与期望一致。

P4 copyByteWithin — 字节提取与写入

三步: ① 提取源字节 $(x \gg (\text{src} * 8)) \& 0xFF$; ② 构造目标字节清零掩码 $\sim(0xFF \ll (\text{dst} * 8))$; ③ 合并 $(x \& \text{clearMask}) | (\text{byte} \ll (\text{dst} * 8))$ 。注意必须先移位再屏蔽——因为 $\gg$ 是算术右移, 会把符号位带入高位。

P5 logicalShift — 逻辑右移

C 的 $\gg$ 对 int 为算术右移, 需事后清除补入的符号位: $(x \gg n) \& \text{mask}$, 其中 mask 低 $32-n$ 位为 1。难点在于 $n=0$ 时 $32-n=32$, 而 32 位机上 $\ll 32$ 是未定义行为。改用 $(1 \ll 31) \gg n \ll 1$ 构造"高 n 位全 1"的掩码: $n>0$ 时正常; $n=0$ 时 0x80000000 $\ll 1$ 溢出得 0, 取反恰为全 1 掩码, 正好满足需要。

P6 swapNibblePairs — 半字节交换

先拼出掩码 0x0F0F0F0F ($t = 0x0F | (0x0F \ll 8)$ 得 0x0F0F, 再 $t | (t \ll 16)$)。然后 $((x \& 10) \ll 4) | ((x \gg 4) \& 10)$ ——低 4 位左移成高 4 位、高 4 位右移成低 4 位。 $x \gg 4$ 的算术右移会污染最高字节, 但紧随其后的 $\& 10$ 恰好将其屏蔽, 体现了「掩码必须置于移位之后」的原则。

P7 secondLowestZeroBit — 找 0 转化为找 1

把"找 0 位"转化为"找 1 位": 令 $t = \sim x$, 则 x 的最低 0 位即 t 的最低 1 位, 而提取最低 1 位有现成写法 $t \& (\sim t + 1)$, 化简后为 $(\sim x) \& (x + 1)$ 。求第二位时, 先把第一个 0 位填上 1 ($x2 = x | z1$), 再在新数上重复同样操作。

P8 oddParity — 异或折叠

把所有位依次异或, 结果即 popcount 的奇偶性 (1 的个数为奇则结果为 1)。用折半折叠代替 31 次异或: 依次 $x \wedge = x \gg 16, \gg 8, \gg 4, \gg 2, \gg 1$, 结果收敛到 bit 0。每步只关心低位, 符号扩展污染的是高位、不参与后续计算, 故算术右移无碍。题目要求偶数个 1 返回 1, 与异或结果相反, 故最后取反。

P9 rotateRightBits — 循环右移

循环右移 = 逻辑右移 m 位 | 左移 $(32-m)$ 位。三个细节：① n 可能 ≥ 32 ，故 $m = n \& 31$ ；② 沿用 P5 的高位掩码技巧处理算术右移污染；③ $m=0$ 时 $32-m=32$ 未定义，改用 $(\sim m + 1) \& 31$ （即 $(-m) \bmod 32$ ）作为左移量——此时左移 0 位，与原值相或仍为原值。

P10 roundEvenPow2 — 舍入到最近偶数倍

设 $q = x \gg n$ （商）、 $half = 2^{(n-1)}$ （半步长），则“是否进位”的条件可统一写成 $r + (q \& 1) > half$ ：余数大于半步时进位；恰好等于半步时仅当商为奇数才进位（进位后商变偶，符合取偶要求）。进一步可把进位直接并入 x ： $((x + half - 1 + (q \& 1)) \gg n) \ll n$ ——因为 $x = q \cdot 2^n + r$ ，除以 2^n 后 floor 恰在 $r + (q \& 1) > half$ 时多给 1。

P11 midpointTowardFirst — 不溢出地求平均

先解决不溢出的平均：由恒等式 $x + y = 2(x \& y) + (x \wedge y)$ 得 $\text{floor}((x+y)/2) = (x \& y) + ((x \wedge y) \gg 1)$ 。当 $x+y$ 为奇数（中点落在两整数之间）且 $x > y$ 时需 +1，使结果偏向 x 。判断 $x > y$ 必须防溢出：同号时用 $(x-y)$ 的符号，异号时 $x > y$ 等价于 $x \geq 0$ ，二者用掩码合并。又因 $x = y$ 时 $x+y$ 必为偶数、bias 自动归零，故可用宽松的 $\geq$ 替代严格的 $>$ ，省下若干操作数。

P12 isBetweenEitherOrder — 符号比较

设 $d1 = x-a$ 、 $d2 = x-b$ ，则“ x 落在区间内”等价于「 $d1$ 与 $d2$ 异号，或其一为 0」。该条件对 a 、 b 的顺序对称，无需分类讨论。减法同样存在 P11 的溢出问题，需分情况取符号。判断相等用 $!d1$ ，它不受回绕影响（ $x-a \equiv 0 \pmod{2^{32}}$ 当且仅当 $x = a$ ）。

P13 mul5Sat — 溢出检测与饱和

“结果与 x 异号即溢出”的判断不可靠——因为 $5x$ 可超出 2^{32} 数倍，回绕后符号不定（如 $x = \text{INT_MAX}$ 时回绕结果仍为正）。故把 $5x$ 拆成 $(x \ll 2) + x$ 分别检测：左移溢出用“右移回来是否还原”判断 $((x \gg 2) \wedge x \neq 0)$ 非零即溢出；加法溢出用经典符号判定（两加数同号而结果异号： $((x \wedge r) \& (x \wedge r)) \gg 31$ ）。饱和值用 $(1 \ll 31) \wedge \sim(x \gg 31)$ 一次算出（ $x > 0$ 得 $0x7FFFFFFF$ ， $x < 0$ 得 $0x80000000$ ），最后用 $r \wedge ((r \wedge \text{sat}) \& \text{ovf})$ 做掩码选择。

P14 classifyAdd3 — 进位追踪

把 x 、 y 、 z 视为无符号数，设 cnt_neg 为负数个数、 carry_out 为无符号三数和的进位，可推出精确和与回绕结果之差为

TEXT

$k = \text{carry_out} - \text{cnt_neg} + (s < 0 ? 1 : 0)$

其中 $k \in \{-1, 0, 1\}$ ，恰为题求答案。无符号加法进位用经典公式 $((a \& b) | ((a | b) \& \sim(a+b))) \gg 31$ ，分两步累加得到 carry_out 。

P15 floatScaleThreeHalves — 浮点乘 1.5

设 $f = M \times 2^{(e-23)}$ ，则 $f \times 1.5 = 3M \times 2^{(e-24)}$ ，新有效数字就是 $3M$ 。因 $3M \in [1.5 \cdot 2^{24}, 3 \cdot 2^{24})$ ，需右移 1 位（若 $< 2^{25}$ ）或 2 位（否则）以归一化，指数相应调整。右移丢弃的低位按 round-to-nearest-even 舍入（用 roundbit 与 sticky 判定）。

另需处理 NaN/Inf（原样返回）、±0（原样返回）、非规格化输入（先循环规格化）与非规格化输出（再次移位舍入）。

P16 floatRoundEven — 浮点舍入到整数

按指数分情况：exp=255 为 NaN/Inf，原样返回；exp<126 时 $|f|<0.5$ ，舍入为 ±0；exp=126 时 $|f|\in[0.5,1)$ ，恰好 0.5 (frac=0) 取偶得 ±0，否则为 ±1.0；exp≥150 时 $|f|\geq 2^{23}$ 已是整数，原样返回；其余需舍入。舍入时取整数部分 $ip = M \gg (23-e)$ 与余数，按 round-half-to-even 决定是否进位，并处理进位到 $2^{(e+1)}$ 的情况。舍入结果为 0 时需保留输入符号。

P17 float_i2f — 整数转浮点

分四步：① x=0 直接返回 0；② 取符号并求绝对值，用无符号取负使得 INT_MIN 也能正确得到 2^{31} ；③ 从高到低找出最高位位置 e；④ 将 24 位有效数字规格化—— $e \leq 23$ 时直接左移补零，否则右移并按 round-to-nearest-even 舍入余数，最后处理进位导致的指数加一。浮点题允许条件与循环，故可直接使用 while 与 if。

P18 bitCount — SWAR 并行统计

采用 SWAR 思路，把一个 32 位寄存器切成多个小计数器并行累加：先用掩码 0x55555555 统计每两位中 1 的个数，再依次合并为 4 位、8 位、16 位计数器，最后合并得到总数。第 3 步写成 $(x + (x \gg 4)) \& 0x0F0F0F0F$ ——先加后屏蔽，因为前两步后每个 4 位块的值不超过 4，相邻两块相加至多 8，不会越界到下一块，故可省去一个屏蔽操作。

P19 bitReverse — 掩码递推

标准做法是五级“分组交换”网络（1、2、4、8、16 位），但把 5 个 32 位掩码从 0-255 的常量拼出来需 16 个操作数，总计 40，超出限额。

关键发现是这些掩码存在递推关系：0x0000FFFF、0x00FF00FF、0x0F0F0F0F、0x33333333、0x55555555 中每一行都是上一行“对折”而来，可用 $mask \hat{=} (mask \ll s)$ 两个操作数从上一个算出下一个（原理：一段连续的 $2L$ 个 1 与自身左移 L 位异或后，恰好裂成两段 L 个 1）。

另一关键：交换顺序可以反转。第 s 级交换实为“位置 i 与 $i \text{ XOR } s$ 互换”，因 XOR 满足交换律，五级彼此可交换，故可按 $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ 的顺序进行，让每个掩码刚造好即使用，省去预先存储的开销。此外半字交换只需 4 个操作数—— $x \ll 16$ 的低 16 位天然为零、移出的高位正是所需丢弃的部分，无需额外屏蔽。最终恰好 34 个操作数，正好卡满限额。

参考资料：

1. 《深入理解计算机系统》（CS:APP，第 3 版）Randal E. Bryant, David R. O'Hallaron 著

第 2 章「信息的表示和处理」——整数补码表示、位运算、IEEE 754 浮点格式

用途：本实验的全部理论依据，尤其是 P3（补码取负 $-x = \sim x + 1$ ）、P15 - P17（浮点格式拆解与舍入规则）

2. Bit Twiddling Hacks

Sean Eron Anderson, Stanford University

「Reverse an N-bit quantity in parallel」章节（含两种变体）

用途：P19 bitReverse。第一种变体是标准的五级交换网络；第二种变体给出了「动态生成掩码」的思路—— $\text{mask} \hat{=} (\text{mask} \ll s)$ 的递推关系正来源于此。

「Counting bits set, in parallel」章节

用途：P18 bitCount 的 SWAR 并行计数思路。

「Compute the integer absolute value (abs) without branching」章节

用途：P3 negativePart 的掩码选择思路。

△ 本项目对该资料的改造说明：原版 P19 解法使用 while 循环，而本实验整数题禁止控制流，故手工展开为 5 轮；同时调整了交换顺序（16→8→4→2→1）以配合掩码的即时构造，并将交换式改写为 $((x \gg s) \& \text{mask}) \mid ((x \& \text{mask}) \ll s)$ 以省去 $\sim \text{mask}$ 的开销。未直接照搬任何代码。